```
%run ../../../Functions/utilities_master_serverless
```

```
spark.conf.set("spark.sql.files.maxPartitionBytes", 134217728)
```

```
bronze_table_name_ibr = "b2b_partner_internal_backend_response_v1"
```

```
endpoint ="CALL_HUNTER_V4"
bronze_table_2 = "pib_call_hunter_v4"
bronze_table_full_2 = f"{catalog_bronze}.{schema_name}.{bronze_table_2}"
bronze_incremental_column = "source_file_modification_timestamp_ist"
bronze_table_name_full = f"{catalog_bronze}.{schema_name}.{bronze_table_name_ibr}"
log_table = f"{catalog_logs}.{schema_name}.incremental_load_table"
Source_bucket_name="s3://b2b-hub-prod/"
print(bronze_table_name_full)
print(bronze_table_full_2)
```

```
date_df = (
    spark.table(log_table)
    .filter(col("Table_Name") == bronze_table_2)
    .filter(col("Layer_Name") == "bronze")
)

if date_df.count() == 0:
    raise ValueError(
        f"No records found in {log_table} for Table_Name = '{bronze_table_2}'"
    )

row = date_df.select("Prev_MaxLoad_ntz", "Curr_MaxLoad_ntz").first()
Curr_MaxLoad_ntz = row["Curr_MaxLoad_ntz"]
Prev_MaxLoad_ntz = row["Prev_MaxLoad_ntz"]

print(Curr_MaxLoad_ntz)
```

```
start_datetime = datetime.now() + timedelta(hours=5, minutes=30)
try:
    df_bronze = (
        spark.read.table(bronze_table_name_full)
        .filter(col("endpoint") == endpoint)
        .filter(col("source_file_modification_timestamp_ist") > Curr_MaxLoad_ntz)
        #.filter((year(col("source_file_modification_timestamp_ist")) == "2025") & (month(col("source_file_modification_timestamp_ist")) > 6) & (month(col("source_file_modification_timestamp_ist")) <= 11))
        )

    source_count = df_bronze.count()
    print(f"Source count: {source_count}")

    if source_count == 0:
        dbutils.notebook.exit("No new data to ingest")
    # display(df_bronze)

except Exception as e:
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    fn_errorlogs(start_datetime, end_datetime, bronze_table_full_2, str(e))
    raise
```

```
df_unwrap = (
    df_bronze.withColumn(
        "payload",
```

7

```python
df_unwrap = (
    df_bronze.withColumn(
        "payload",
        try_parse_json(
            dynamodb_json_to_standard_json_udf(to_json(col("payload")).cast("string"))
        ),
    )
)
# display(df_unwrap.limit(10))
```

8

```python
df_bronze_with_paths = df_unwrap.withColumn(
    "response_body_path",
    when(
        expr(f"variant_get(payload, '$.NewImage.response_body', 'String')").rlike(
            r"^/?[a-zA-Z0-9_\-/\.]+$"
        ),
        concat(
            lit(Source_bucket_name),
            expr(f"variant_get(payload, '$.NewImage.response_body', 'String')"),
        ),
    ).otherwise(lit(None)),
).withColumn(
    "request_body_path",
    when(
        expr(f"variant_get(payload, '$.NewImage.request_body', 'String')").rlike(
            r"^/?[a-zA-Z0-9_\-/\.]+$"
        ),
        concat(
            lit(Source_bucket_name),
            expr(f"variant_get(payload, '$.NewImage.request_body', 'String')"),
        ),
    ).otherwise(lit(None)),
)
# display(df_bronze_with_paths.limit(10))
```

9

```python
request_body_paths_df = df_bronze_with_paths.select("request_body_path").distinct().filter(col("request_body_path").isNotNull())

print("Request Files:", request_body_paths_df.count())
# Repartition for parallelism before collect
request_body_paths_df = request_body_paths_df.repartition(8)

# Use Spark to check file existence in bulk, avoiding Python loops
request_paths = [row.request_body_path for row in request_body_paths_df.collect()]
df_request_body_file = (
    spark.read.format("binaryFile")
    .option("pathGlobFilter", "*")
    .load(request_paths)
    .repartition(8)
    .select(
        col("path").alias("request_file_path"),
        col("content").cast("string").alias("request_body"),
        from_utc_timestamp(
            col("modificationTime"),
            "Asia/Kolkata"
        ).cast("timestamp_ntz").alias("request_file_modification_timestamp_ist")
    )
)

df_request_body_file = df_request_body_file.withColumn(
    "request_body",
    from_json(
        col("request_body"),
        MapType(StringType(), StringType())
    )
)

# Identify not found paths by anti-join
found_paths_df = df_request_body_file.select("request_file_path").distinct()
not_found_paths_df = request_body_paths_df.join(
    found_paths_df,
    request_body_paths_df.request_body_path == found_paths_df.request_file_path,
    "left_anti"
).select(col("request_body_path").alias("request_file_path"))

if not_found_paths_df.count() > 0:
    not_found_df = not_found_paths_df.withColumn("request_body", create_map(lit("error"), lit("file not found"))) \
        .withColumn("request_file_modification_timestamp_ist", lit(None).cast("timestamp_ntz"))
```

9

```python
if not_found_paths_df.count() > 0:
    not_found_df = not_found_paths_df.withColumn("request_body", create_map(lit("error"), lit("file not found"))) \
        .withColumn("request_file_modification_timestamp_ist", lit(None).cast("timestamp_ntz"))
    df_request_body_file = df_request_body_file.unionByName(not_found_df).repartition(8)

if df_request_body_file.isEmpty():
    df_request_body_file = spark.createDataFrame([], schema="request_file_path STRING, request_body MAP<STRING,STRING>, request_file_modification_timestamp_ist timestamp_ntz")

# display(df_request_body_file)
```

10

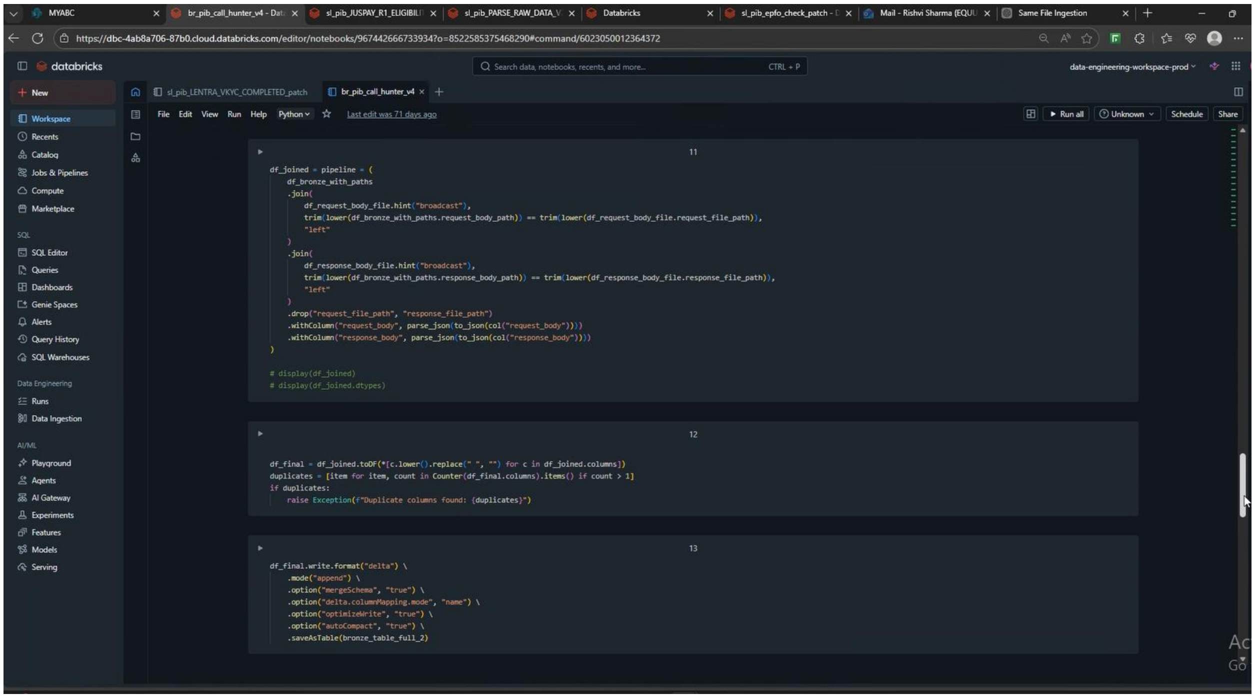
```python
response_body_paths_df = df_bronze_with_paths.select("response_body_path").distinct().filter(col("response_body_path").isNotNull())

print("Response Files:", response_body_paths_df.count())
# Repartition for parallelism before collect
response_body_paths_df = response_body_paths_df.repartition(8)

# Use Spark to check file existence in bulk, avoiding Python loops
response_paths = [row.response_body_path for row in response_body_paths_df.collect()]
df_response_body_file = (
    spark.read.format("binaryFile")
    .option("pathGlobFilter", "*")
    .load(response_paths)
    .repartition(8)
    .select(
        col("path").alias("response_file_path"),
        col("content").cast("string").alias("response_body"),
        from_utc_timestamp(
            col("modificationTime"),
            "Asia/Kolkata"
        ).cast("timestamp_ntz").alias("response_file_modification_timestamp_ist")
    )
)

df_response_body_file = df_response_body_file.withColumn(
    "response_body",
    from_json(
        col("response_body"),
        MapType(StringType(), StringType())
    )
```

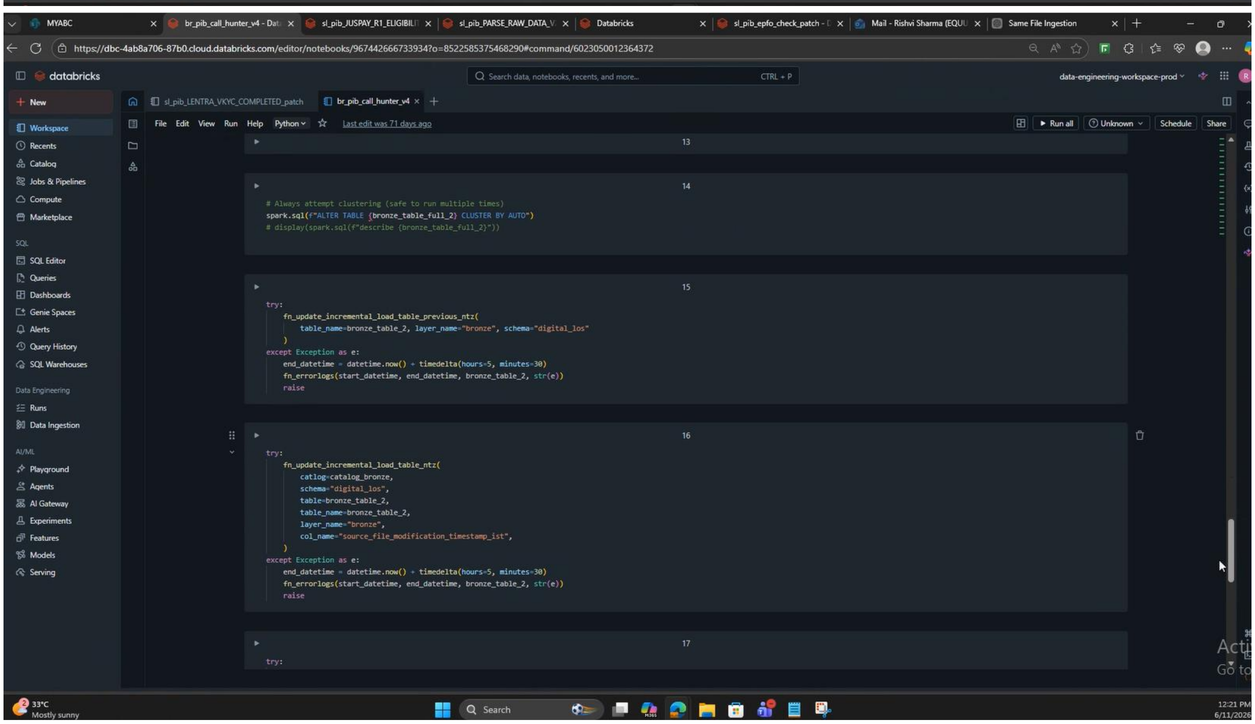
10

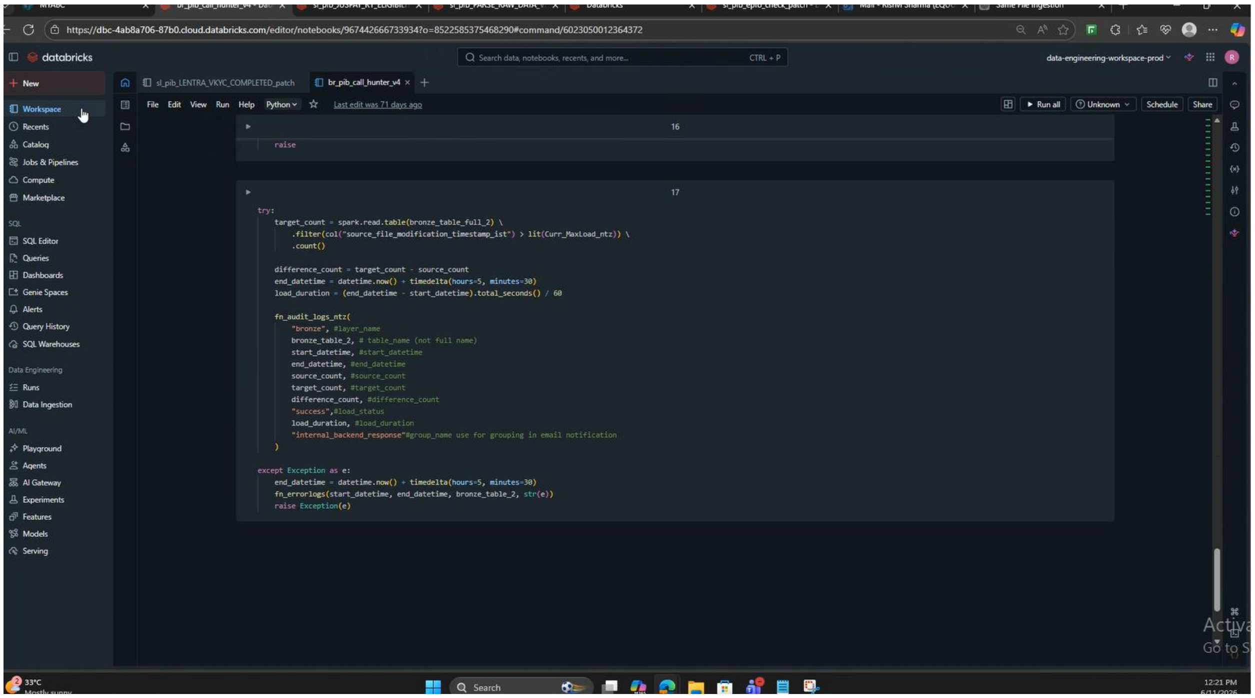
```python
df_response_body_file = df_response_body_file.withColumn(
    "response_body",
    from_json(
        col("response_body"),
        MapType(StringType(), StringType())
    )
)

# Identify not found paths by anti-join
found_response_paths_df = df_response_body_file.select("response_file_path").distinct()
not_found_response_paths_df = response_body_paths_df.join(
    found_response_paths_df,
    response_body_paths_df.response_body_path == found_response_paths_df.response_file_path,
    "left_anti"
).select(col("response_body_path").alias("response_file_path"))

if not_found_response_paths_df.count() > 0:
    not_found_response_df = not_found_response_paths_df.withColumn("response_body", create_map(lit("error"), lit("file not found"))) \
        .withColumn("response_file_modification_timestamp_ist", lit(None).cast("timestamp_ntz"))
    df_response_body_file = df_response_body_file.unionByName(not_found_response_df).repartition(8)

if df_response_body_file.isEmpty():
    df_response_body_file = spark.createDataFrame([], schema="response_file_path STRING, response_body MAP<STRING,STRING>, response_file_modification_timestamp_ist timestamp_ntz")

# display(df_response_body_file)
```

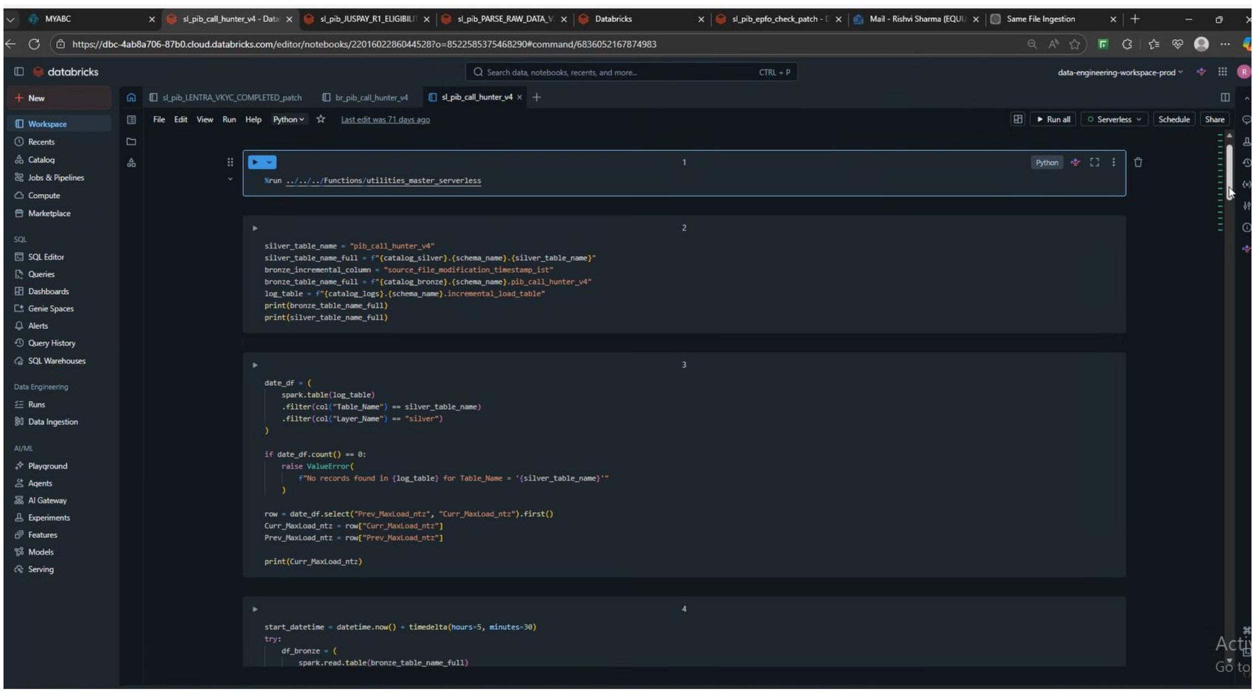
11

```python
df_joined = pipeline = (
    df_bronze_with_paths
    .join(
        df_request_body_file.hint("broadcast"),
        trim(lower(df_bronze_with_paths.request_body_path)) == trim(lower(df_request_body_file.request_file_path)),
        "left"
    )
    .join(
        df_response_body_file.hint("broadcast"),
        trim(lower(df_bronze_with_paths.response_body_path)) == trim(lower(df_response_body_file.response_file_path)),
        "left"
    )
```


https://dbc-4ab8a706-87b0.cloud.databricks.com/editor/notebooks/967442666733934?o=8522585375468290#command/6023050012364372
databricks
sl_pib_LENTRA_VKYC_COMPLETED_patch
br_pib_call_hunter_v4
Last edit was 71 days ago
11
df_joined = pipeline = (
    df_bronze_with_paths
    .join(
        df_request_body_file.hint("broadcast"),
        trim(lower(df_bronze_with_paths.request_body_path)) == trim(lower(df_request_body_file.request_file_path)),
        "left"
)
    .join(
        df_response_body_file.hint("broadcast"),
        trim(lower(df_bronze_with_paths.response_body_path)) == trim(lower(df_response_body_file.response_file_path)),
        "left"
)
    .drop("request_file_path", "response_file_path")
    .withColumn("request_body", parse_json(to_json(col("request_body"))))
    .withColumn("response_body", parse_json(to_json(col("response_body"))))
)
display(df_joined)
display(df_joined.dtypes)
12
df_final = df_joined.toDF(*[c.lower().replace(" ", "") for c in df_joined.columns])
duplicates = [item for item, count in Counter(df_final.columns).items() if count > 1]
if duplicates:
    raise Exception(f"Duplicate columns found: {duplicates}")
13
df_final.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .option("delta.columnMapping.mode", "name") \
    .option("optimizeWrite", "true") \
    .option("autoCompact", "true") \
    .saveAsTable(bronze_table_full_2)


https://dbc-4ab8a706-87b0.cloud.databricks.com/editor/notebooks/967442666733934?o=8522585375468290#command/6023050012364372
databricks
sl_pib_LENTRA_VKYC_COMPLETED_patch
br_pib_call_hunter_v4
Last edit was 71 days ago
13
14
# Always attempt clustering (safe to run multiple times)
spark.sql(f"ALTER TABLE {bronze_table_full_2} CLUSTER BY AUTO")
# display(spark.sql(f"describe {bronze_table_full_2}"))
15
try:
    fn_update_incremental_load_table_previous_ntz(
        table_name=bronze_table_2, layer_name="bronze", schema="digital_los"
)
except Exception as e:
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    fn_errorlogs(start_datetime, end_datetime, bronze_table_2, str(e))
    raise
16
try:
    fn_update_incremental_load_table_ntz(
        catlog=catalog_bronze,
        schema="digital_los",
        table=bronze_table_2,
        table_name=bronze_table_2,
        layer_name="bronze",
        col_name="source_file_modification_timestamp_ist",
)
except Exception as e:
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    fn_errorlogs(start_datetime, end_datetime, bronze_table_2, str(e))
    raise
17
try:
12:21 PM
6/11/2026


https://dbc-4ab8a706-87b0.cloud.databricks.com/editor/notebooks/967442666733934?o=8522585375468290#command/6023050012364372
databricks
sl_pib_LENTRA_VKYC_COMPLETED_patch
br_pib_call_hunter_v4
File Edit View Run Help Python
Last edit was 71 days ago
Run all
Unknown
Schedule
Share
16
raise
17
try:
target_count = spark.read.table(bronze_table_full_2) \
.filter(col("source_file_modification_timestamp_ist") > lit(Curr_MaxLoad_ntz)) \
.count()
difference_count = target_count - source_count
end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
load_duration = (end_datetime - start_datetime).total_seconds() / 60
fn_audit_logs_ntz(
"bronze", #layer_name
bronze_table_2, # table_name (not full name)
start_datetime, #start_datetime
end_datetime, #end_datetime
source_count, #source_count
target_count, #target_count
difference_count, #difference_count
"success",#load_status
load_duration, #load_duration
"internal_backend_response"#group_name use for grouping in email notification
)
except Exception as e:
end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
fn_errorlogs(start_datetime, end_datetime, bronze_table_2, str(e))
raise Exception(e)


https://dbc-4ab8a706-87b0.cloud.databricks.com/editor/notebooks/2201602286044528?o=8522585375468290#command/6836052167874983
databricks
sl_pib_LENTRA_VKYC_COMPLETED_patch
br_pib_call_hunter_v4
sl_pib_call_hunter_v4
File Edit View Run Help Python
Last edit was 71 days ago
Run all
Serverless
Schedule
Share
1
%run ../../../Functions/utilities_master_serverless
2
silver_table_name = "pib_call_hunter_v4"
silver_table_name_full = f"{catalog_silver}.{schema_name}.{silver_table_name}"
bronze_incremental_column = "source_file_modification_timestamp_ist"
bronze_table_name_full = f"{catalog_bronze}.{schema_name}.pib_call_hunter_v4"
log_table = f"{catalog_logs}.{schema_name}.incremental_load_table"
print(bronze_table_name_full)
print(silver_table_name_full)
3
date_df = (
spark.table(log_table)
.filter(col("Table_Name") == silver_table_name)
.filter(col("Layer_Name") == "silver")
)
if date_df.count() == 0:
raise ValueError(
f"No records found in {log_table} for Table_Name = '{silver_table_name}'"
)
row = date_df.select("Prev_MaxLoad_ntz", "Curr_MaxLoad_ntz").first()
Curr_MaxLoad_ntz = row["Curr_MaxLoad_ntz"]
Prev_MaxLoad_ntz = row["Prev_MaxLoad_ntz"]
print(Curr_MaxLoad_ntz)
4
start_datetime = datetime.now() + timedelta(hours=5, minutes=30)
try:
df_bronze = (
spark.read.table(bronze_table_name_full)

```python
keys_payload_values = keys["payload_keys"]


# Step 3: Extract values using variant_get
p_cols = [
    expr(f"variant_get(payload, '$.{k}', 'VARIANT')").alias(f"{k}")
    for k in keys_payload_values
]


# Step 4: Select expanded columns
df_split = df_bronze.select("*", *p_cols)

# Step 5: Drop unwanted columns
df_split = df_split.drop(
    "payload",
)
```

```python
# Step 1: Extract all dynamic keys
df_keys_2 = df_split.select(
    explode(map_keys(col("NewImage").cast("map<string, variant>"))).alias("NewImage_key"),
    explode(map_keys(col("Metadata").cast("map<string, variant>"))).alias("Metadata_key"),
    explode(map_keys(col("Keys").cast("map<string, variant>"))).alias("Keys_key"),
)

# Step 2: Collect distinct keys
keys_2 = df_keys_2.agg(
    collect_set("NewImage_key").alias("NewImage_keys"),
    collect_set("Metadata_key").alias("Metadata_keys"),
    collect_set("Keys_key").alias("Keys_keys"),
).collect()[0]

keys_NewImage_values = keys_2["NewImage_keys"]
keys_Metadata_values = keys_2["Metadata_keys"]
keys_Keys_values = keys_2["Keys_keys"]

# Step 3: Extract values using variant_get
key_cols = [
```

```python
# Step 3: Extract values using variant_get
key_cols = [
    expr(f"variant_get(Keys, '$.{k}', 'VARIANT')").alias(f"key_{k}")
    for k in keys_Keys_values
]

md_cols = [
    expr(f"variant_get(Metadata, '$.{k}', 'VARIANT')").alias(f"md_{k}")
    for k in keys_Metadata_values
]

ni_cols = [
    expr(f"variant_get(NewImage, '$.{k}', 'VARIANT')").alias(f"ni_{k}")
    for k in keys_NewImage_values
]

# Step 4: Select expanded columns
df_split_2 = df_split.select("*", *key_cols, *md_cols, *ni_cols)

# Step 5: Drop unwanted columns
df_split_2 = df_split_2.drop(
    "Keys",
    "Metadata",
    "NewImage",
    "OldImage",
    "ni_response_body",   # same as response_body_path
    "ni_request_body",    # same as request_body_path
    "ni_endpoint", # same as endpoint
    "ni_application_id", #same as application_id
)
```

```python
def ensure_nullable_column(df, src_col=None, target_col=None, dtype="string"):
    if src_col and src_col in df.columns:
        return df.withColumn(target_col, col(src_col).cast(dtype))
    else:
        return df.withColumn(target_col, lit(None).cast(dtype))
```

8

```python
df_split_2 = ensure_nullable_column(
    df=df_split_2,
    src_col="ni_status",
    target_col="ni_status",
    dtype="variant"
)
```

9

```python
df_final = (
    df_split_2.withColumn(
        "inserted_timestamp_ist",
        from_utc_timestamp(current_timestamp(), "Asia/Kolkata").cast("timestamp_ntz"),
    )
    .withColumn(
        "bronze_inserted_timestamp_ist",
        col("inserted_timestamp_ist").cast("timestamp_ntz"),
    )
    .selectExpr(
        "application_id",
        "partner_journey_id",
        "endpoint",
        "ni_status::string",
        "source_file_modification_timestamp_ist",  # file creation time
        "bronze_inserted_timestamp_ist",  # bronze time
        "inserted_timestamp_ist",  # silver time
        "* EXCEPT(application_id, partner_journey_id, endpoint, ni_status, source_file_modification_timestamp_ist,bronze_inserted_timestamp_ist,inserted_timestamp_ist)",
    )
)
df_final = df_final.withColumnsRenamed(
    {col: col.replace("ni_", "") for col in df_final.columns if col.startswith("ni_")}
)
```

10

```python
df_final = ensure_nullable_column(
    df=df_final,
    src_col="response_body",
    target_col="response_body",
    dtype="variant"
)
df_final = ensure_nullable_column(
    df=df_final,
    src_col="request_body",
    target_col="request_body",
    dtype="variant"
)
```

11

```python
df_final = df_final.withColumn(
    "response_body",
    expr("""
        CASE
            WHEN response_body IS NULL THEN NULL
            WHEN try_parse_json(regexp_replace(response_body, '\\\\"', '\"')) IS NULL
            THEN parse_json(
                to_json(
                    named_struct(
                        'type', 'HTML',
                        'content', regexp_replace(response_body, '\\\r\\\\n', '')
                    )
                )
            )
            ELSE parse_json(regexp_replace(response_body, '\\\\"', '\"'))
        END
    """)
).withColumn(
    "request_body",
    expr("""
        CASE
            WHEN request_body IS NULL THEN NULL
            WHEN try_parse_json(regexp_replace(request_body, '\\\\"', '\"')) IS NULL
            THEN parse_json(
                to_json(
```

11

```python
        "request_body",
        expr("""
            CASE
                WHEN request_body IS NULL THEN NULL
                WHEN try_parse_json(regexp_replace(request_body, '\\\\\"', '\"')) IS NULL
                THEN parse_json(
                    to_json(
                        named_struct(
                            'type', 'HTML',
                            'content', regexp_replace(request_body, '\\\r\\\n', '')
                        )
                    )
                )
                ELSE parse_json(regexp_replace(request_body, '\\\\"', '\"'))
            END
        """)
    )
```

12

```python
# Step 1: Get all top-level keys from response_body
all_keys = (
    df_final.filter("response_body IS NOT NULL")
    .select(explode(map_keys(col("response_body").cast("map<string, variant>"))).alias("key"))
    .agg(collect_set("key").alias("keys"))
    .collect()[0]["keys"]
)

# Step 2: Dynamically detect fields that are STRING-type variants containing valid JSON objects/arrays
open_brace = chr(123)  # '{' — avoids f-string escaping issues
check_exprs = [
    expr(f"""
        MAX(CASE
            WHEN schema_of_variant(response_body:`{k}`) = 'STRING'
                AND LEFT(CAST(response_body:`{k}` AS STRING), 1) IN ('{open_brace}', '[')
                AND try_parse_json(CAST(response_body:`{k}` AS STRING)) IS NOT NULL
            THEN 1 ELSE 0 END
        ) as `{k}`
    """)
    for k in all_keys
]
```

12

```python
result = df_final.filter("response_body IS NOT NULL").select(*check_exprs).collect()[0]
json_string_fields = [k for k in all_keys if result[k] == 1]
print(f"Detected stringified JSON fields: {json_string_fields}")

# Step 3: Parse each stringified JSON field into proper JSON within response_body
for field in json_string_fields:
    df_final = df_final.withColumn(
        "response_body",
        expr(f"""
            CASE
                WHEN response_body IS NULL THEN NULL
                WHEN response_body:`{field}` IS NOT NULL
                    AND try_parse_json(CAST(response_body:`{field}` AS STRING)) IS NOT NULL THEN
                    parse_json(to_json(map_concat(
                        map_filter(
                            CAST(response_body AS MAP<STRING, VARIANT>),
                            (k, v) -> k != '{field}'
                        ),
                        map(
                            '{field}',
                            try_parse_json(CAST(response_body:`{field}` AS STRING))
                        )
                    )))
                ELSE response_body
            END
        """)
    )
```

13

```python
# display(df_final.dtypes)
df_final = df_final.toDF(*[c.lower().replace(" ", "") for c in df_final.columns])
duplicates = [item for item, count in Counter(df_final.columns).items() if count > 1]
if duplicates:
    raise Exception(f"Duplicate columns found: {duplicates}")
```

14

```python
df_final.write.format("delta") \
```

13

```python
# display(df_final.dtypes)
df_final = df_final.toDF(*[c.lower().replace(" ", "") for c in df_final.columns])
duplicates = [item for item, count in Counter(df_final.columns).items() if count > 1]
if duplicates:
    raise Exception(f"Duplicate columns found: {duplicates}")
```

14

```python
df_final.write.format("delta") \
.mode("append") \
.option("mergeSchema", "true") \
.option("delta.columnMapping.mode", "name") \
.saveAsTable(silver_table_name_full)

# Always attempt clustering (safe to run multiple times)
spark.sql(f"ALTER TABLE {silver_table_name_full} CLUSTER BY AUTO")
```

15

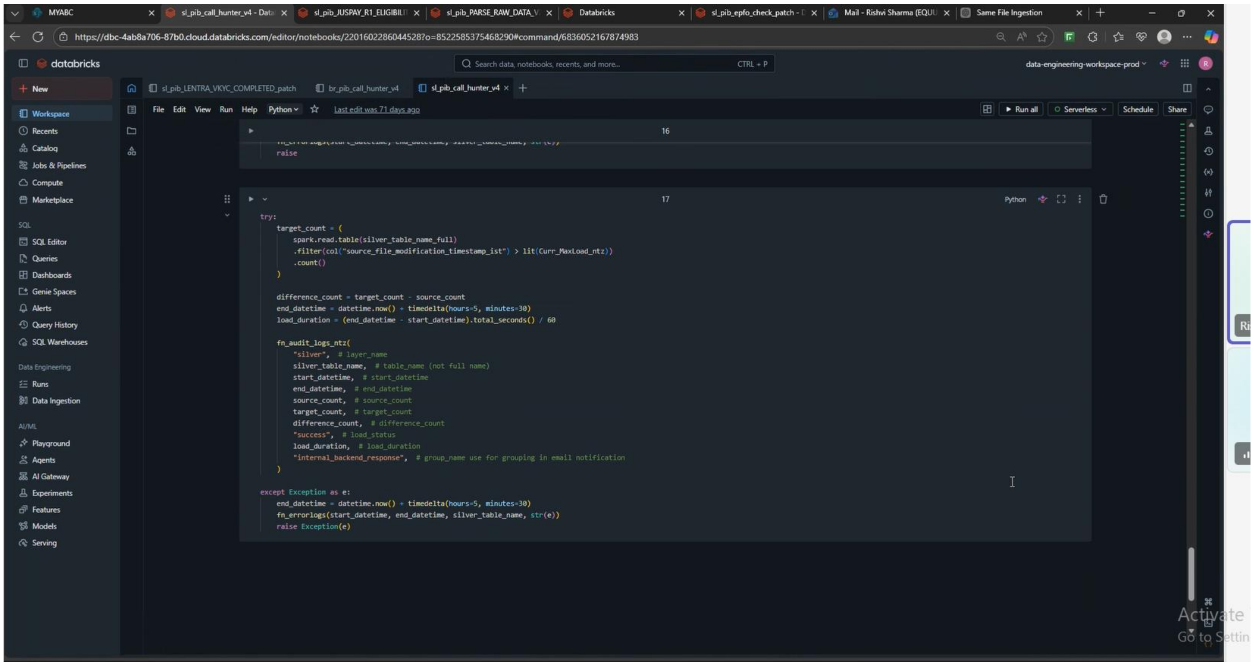
```python
try:
    fn_update_incremental_load_table_previous_ntz(
        table_name=silver_table_name,
        layer_name="silver",
        schema="digital_los"
    )
except Exception as e:
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    fn_errorlogs(start_datetime, end_datetime, silver_table_name, str(e))
    raise
```

16

```python
try:
    fn_update_incremental_load_table_ntz(
    catlog=catalog_silver,
    schema="digital_los",
    table=silver_table_name,
    table_name=silver_table_name,
    layer_name="silver",
    col_name="source_file_modification_timestamp_ist"
)
except Exception as e:
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    fn_errorlogs(start_datetime, end_datetime, silver_table_name, str(e))
    raise
```

17

```python
try:
    target_count = (
        spark.read.table(silver_table_name_full)
        .filter(col("source_file_modification_timestamp_ist") > lit(Curr_MaxLoad_ntz))
        .count()
    )

    difference_count = target_count - source_count
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    load_duration = (end_datetime - start_datetime).total_seconds() / 60

    fn_audit_logs_ntz(
        "silver",  # layer_name
        silver_table_name,  # table_name (not full name)
        start_datetime,  # start_datetime
        end_datetime,  # end_datetime
        source_count,  # source_count
        target_count,  # target_count
        difference_count,  # difference_count
        "success",  # load_status
        load_duration,  # load_duration
        "internal_backend_response",  # group_name use for grouping in email notification
    )

except Exception as e:
```



sl_pib_call_hunter_v4
https://dbc-4ab8a706-87b0.cloud.databricks.com/editor/notebooks/2201602286044528?o=8522585375468290#command/6836052167874983
databricks
Search data, notebooks, recents, and more...
sl_pib_LENTRA_VKYC_COMPLETED_patch
br_pib_call_hunter_v4
File Edit View Run Help Python
Last edit was 71 days ago
Run all
Serverless
Schedule
Share
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Spaces
Alerts
Query History
SQL Warehouses
Data Engineering
Runs
Data Ingestion
AI/ML
Playground
Agents
AI Gateway
Experiments
Features
Models
Serving
16
raise
17
try:
    target_count = (
        spark.read.table(silver_table_name_full)
        .filter(col("source_file_modification_timestamp_ist") > lit(Curr_MaxLoad_ntz))
        .count()
)

    difference_count = target_count - source_count
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    load_duration = (end_datetime - start_datetime).total_seconds() / 60

    fn_audit_logs_ntz(
        "silver", # layer_name
        silver_table_name, # table_name (not full name)
        start_datetime, # start_datetime
        end_datetime, # end_datetime
        source_count, # source_count
        target_count, # target_count
        difference_count, # difference_count
        "success", # load_status
        load_duration, # load_duration
        "internal_backend_response", # group_name use for grouping in email notification
)

except Exception as e:
    end_datetime = datetime.now() + timedelta(hours=5, minutes=30)
    fn_errorlogs(start_datetime, end_datetime, silver_table_name, str(e))
    raise Exception(e)